Smiley Quest

🤪Game Design Document🥲

1. Vision

Smiley Quest is a modern reinterpretation of the Pac-Man Game, blending fast arcade gameplay with light combat, collectible weapons, smiley-head characters, and a modular progression system. The game aims to deliver immediate fun, replayability, and charming simplicity while integrating modern features such as character customization, scoring rules, and structured screen management.

1.1 Goals

- Deliver a nostalgic arcade experience enhanced with modern systems (weapon abilities, enemy AI, customizable avatars).
- Provide highly readable gameplay with fast input response and visually expressive characters.
- Maintain long-term replayability through level variety, scoring systems, and progressive difficulty.
- Integrate robust Java OOP principles throughout the architecture (Encapsulation, Abstraction, Inheritance, Polymorphism).

1.2 Technical Goals

- Build a clean and extensible entity system for Player, Enemies, Items, and UI.
- Implement a modular **Screen Management System** with multiple scenes (Splash, Menu, Game, Pause, Settings, Game Over).
- Provide maintainable subsystems: Audio, Scoring, Timer, Enemy Rendering, Events.

2. Target Audience

- Casual players and retro-gaming fans (6+).
- Players who enjoy micro-challenges, short sessions, and character customization.
- Platform: **PC**.

3. Core Gameplay

3.1 Core Loop

- Navigate a maze-like level.
- Collect weapons and items.
- Defeat or avoid enemies.
- Find the **golden chest** to progress to the next level.
- Survive increasing waves of danger as levels become harder.

3.2 Player Controls (PC)

- **Movement:** Arrow Keys
- **Action / Attack:** Space
- **Pause:** Escape
- **Menu**: M

3.3 Player State & Progression

- **Lives:** 3 per run (extra life may be awarded between levels).
- **Health & Damage:** Based on equipped weapons and enemy types.
- **Score:**
    - Time-based scoring (via **`TimeBasedScoreRule`**)
    - Enemy kills
    - Level completion bonus
- **Weapons:** Temporary power-ups granting special effects (sword, axe, bow, poison etc.).
- **Difficulty Scaling:**

- Enemy health increases
- Enemy count increases
- Faster AI behaviors
- Tighter mazes and fewer power-ups

4. Player Character & Customization

4.1 Smiley Heads

Players choose from multiple **smiley faces**, each offering:

4.2 Character Selection System

Handled in **CharacterSelectScreen**:

- Displays available smileys
- Stores selection in **`selectedSmileyIndex`**
- Used by the game world during initialization

5. Ennemis

Enemies use specific sprites and are rendered by the **`EnemyRenderer`** system. Each enemy has unique movement patterns, behaviors, speeds, and attack styles. All the enemies have a cooldown system for their attacks.

5.1 GhostEnemy

- **Behavior:**
  - Diagonal movement
  - Slight wall phasing (~30% of collisions ignored)
  - Variable speed spikes

- **Move Logic:**
  - Soft interpolation toward the player

5.2 LarvaEnemy

- **Behavior:**
  - Slow
  - Pursues player only within detection radius
- **Move Logic:**
  - If distance < 180px → move toward player

5.3 SkeletonWeakEnemy

- **Behavior:**
  - Straightforward pursuit
  - Halts to attack at intervals
  - Medium speed and damage

- **Move Logic:**
  - Direct pathing

5.4 SkeletonStrongEnemy

- **Behavior:**
  - Faster
  - Performs lateral dodges

5.5 DragonEnemy (Mini-Boss – From Level 3-4)

- **Behavior:**
  - Limited movement
  - Higher health

- **Move Logic:**
  - Rotates around player
  - Switches direction every 2 seconds

5.6 FinalBossEnemy (Baby Boss)

- **Behavior:**
  - Very fast dash
  - Area and line attacks
  - Higher health
  - Inflicts higher damage to the player

- **Move Logic: Sequential Pattern**
  - Rush
  - Retreat
  - Heavy attack

6. Level Design & Progression

- **4 progressive levels** included in the base game.

- Levels vary by:

    - Maze geometry
    - Corridor width
    - Room structure
    - Enemy number and type
      Weapon availability

Difficulty Elements

- Faster enemies
- More complex patterns
- Stronger item placement logic

7. Power-ups & items

Weapons differ by damage and temporary abilities. Weapons give score bonus to the player.

8. User Interface

8.1 HUD

- Lives
- Score
- Weapon icon + cooldown

8.2 Screens (based on ScreenManager)

- **SplashScreen** (intro animation using TextureAtlas)
- **MenuScreen**
- **CharacterSelectScreen**
- **GameScreen**
- **PausedScreen**
- **SettingsScreen** (audio, controls)
- **GameOverScreen**
- **GameWinnerScreen**

Scene transitions are managed through the **`ScreenManager`** and **`GameState`**.

9. Audio

9.1 Music

- Retro Pac-Man chiptune soundtrack.
- Looping background music controlled by **`AudioManager`.**

AudioManager handles:

- Mute/unmute
- Persistent settings

10. Artistic Direction

- Bright, expressive smiley faces
- Cartoon-like enemies
- Color-coded weapons
- Minimalist UI with bold shapes

11. Technology Stack

- **Language:** Java
- **Framework:** LibGDX
- **Rendering:** SpriteBatch, TextureAtlas
- **IDE:** IntelliJ
- **Core Systems:**
  - ScreenManager
  - AudioManager
  - ScoreSystem
  - EnemyRenderer
  - GameTimer
  - Observable / Observer event system

12. High-Level Architecture

12.1 Scene System

Based on LibGDX **`Screen`** interface:

- show / update / render / resize / pause / resume / hide / dispose

12.2 GameWorld

- Holds player, enemies, items, collisions, level logic
- Core loop interacts with Scoring and Timer systems

12.3 Event System

- `Observable` + `Observer` interfaces
- Events communicated through `GameEvent`
- Useful for scoring, UI updates, audio triggers

12.4 Scoring System

- Flexible rules using **polymorphism**
- Event-based scoring planned

13. Asset List

- **Smiley heads:** 7
- **Enemies:** 5 types with unique sprites
- **Weapons:** sword, axe, bow, mace, poison icons
- **Tilemaps:** 4 levels
- **User Interface:** buttons, panels, fonts
- **Audio:** retro Pac-Man background music

14. Future Improvements

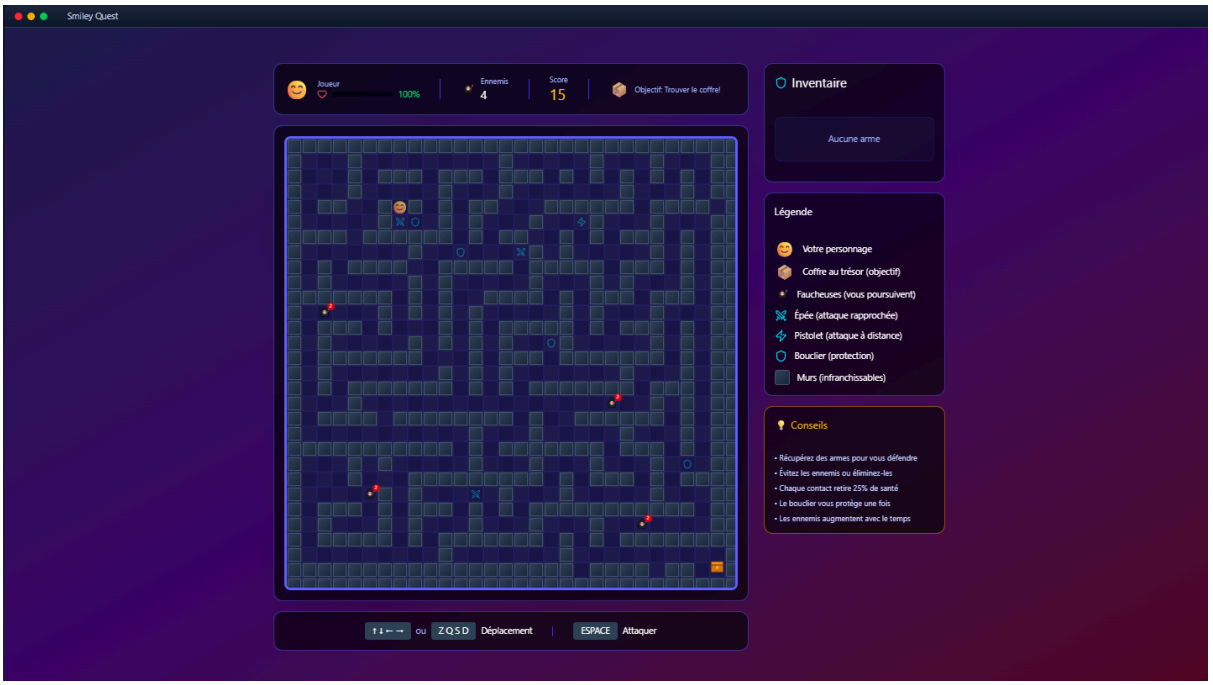
- Community level editor
- RPG progression system
- Weapon crafting or upgrades
- Achievements, challenges (Timer, scoring objective, kill streaks), seasonal modes

Conclusion

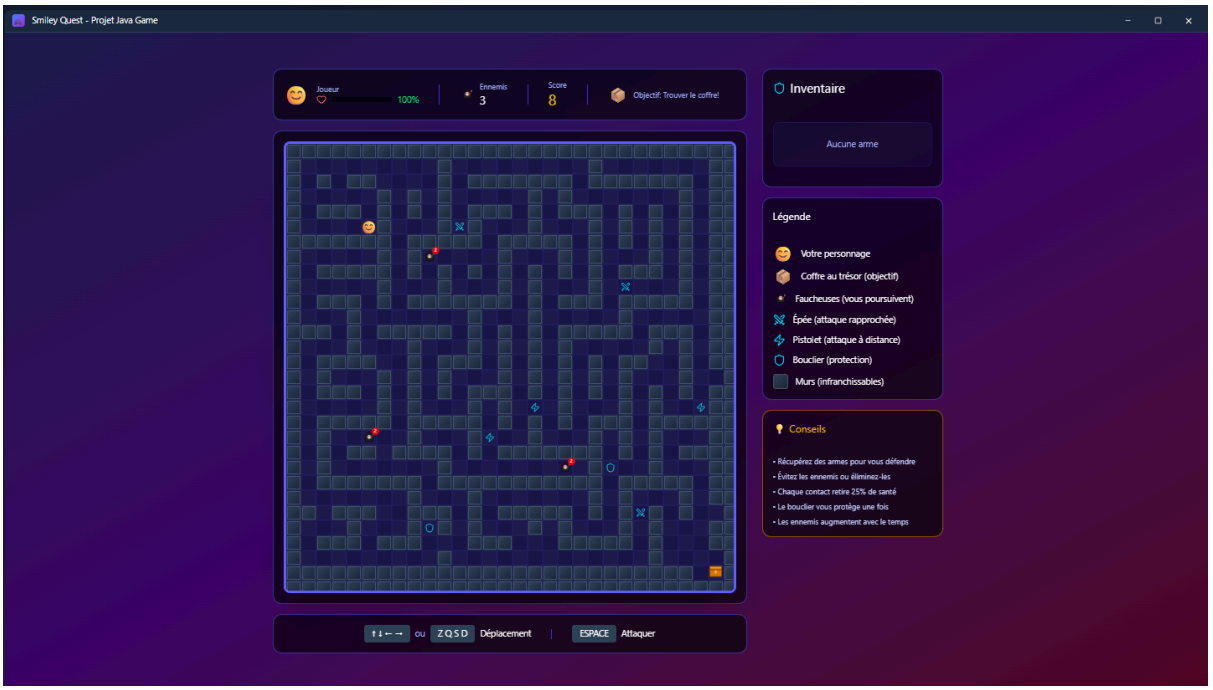
Smiley Quest blends arcade heritage with modern customization, expressive characters, enemy diversity, and modular gameplay systems.

MOCK : Visualisation du jeu sur PC

MacOS



Windows :



GAME MOODBOARD

